

Coda-P Library

MANPAGES

LIBRARY

coda-p
#include <coda-p.h>

NAME

Array_ToData — create Data from array of raw object data based on sizeO(element)

SYNOPSIS

Data
Array_ToData(Array array);

DESCRIPTION

Combine the raw data for the objects in an array into a blob of data. Note: Char objects may contain extra bytes after the null and Os("string") objects are not supported!

RETURN VALUES

Will return NULL for empty arrays.

NAME

Array_sort — sort an Array

SYNOPSIS

void
Array_sort(Array array, void *IcmpfunVVC, void *context);

DESCRIPTION

Sort an array with: int (*cmpfun)(Obj a,Obj b,void* context).

EXAMPLES

Array_sort(list,strcmp,0);

SEE ALSO

pointer_sort(3)

NAME

Bool_Value — create a Bool object

SYNOPSIS

Bool
Bool_Value(bool value);

DESCRIPTION

Create a new Bool object with a single value.

NAME

Char_Split — split Char string into an Array \$S

SYNOPSIS

Array
Char_Split(*char* string, char* delimiter*);

DESCRIPTION

Returns an Array of Char strings generated by splitting a Char string at a delimiter and removing the delimiters.

NAME

Data_FromFile — Data from file contents or NULL

SYNOPSIS

Data
Data_FromFile(*char *path*);

DESCRIPTION

Returns Data by reading the entire contents of "file", or NULL. If NULL see OError() for details.

NAME

Data_NewBlock — create a Data object

SYNOPSIS

Data
Data_NewBlock(*Data self, int count, pointer address*);

DESCRIPTION

Returns a new Data object from the supplied address. Note: If count is less than 1, NULL is returned. The prototype argument is not used.

NAME

Data_base64decode — decode base64 data

SYNOPSIS

int
Data_base64decode(*char *dp, int dlen, char *sp, int slen*);

DESCRIPTION

Decode base64 printable characters 'A-Za-z0-9+/' to bytes. Returns the number of bytes decoded or a negative value for a error. Note: source(sp,slen) and destination(dp,dlen). Return nout or -1 bad char, -2 too big

NAME

Data_base64encode — encode base64 data

SYNOPSIS

int

Data_base64encode(*char *dp, int dlen, char *sp, int slen, int addpad*);

DESCRIPTION

Encode bytes to base64 printable characters.'A-Za-z0-9+/'. Returns the number of bytes encoded or a negative value for a error. If 'addpad' then one or two '=' characters may pad the end for output in multiples of four bytes. Note: source(sp,slen) and destination(dp,dlen) Return(nout) or -2 too big.

NAME

DateString_Value — create a DateString object

SYNOPSIS

DateString

DateString_Value(*char *string*);

DESCRIPTION

Returns a new DateString object with a copy of 'string'. Note: Objects created with this function will have a minimum of 32 bytes of storage to allow for reformatting.

NAME

FileMem_Open — open a FileMem with optional preloaded data

SYNOPSIS

FileMem

FileMem_Open(*Data optional*);

DESCRIPTION

Returns a new FileMem for read and/or write to memory.

NAME

FileMem_ToData — capture memory output //

SYNOPSIS

Data

FileMem_ToData(*FileMem self*);

DESCRIPTION

Returns the Data written to memory or NULL.

NAME

FileMem_array — get internal Array or NULL

SYNOPSIS

Array
FileMem_array(*FileMem self*);

DESCRIPTION

Returns the Array for the current stream.

NAME

FileMem_oPrintf — printf style //

SYNOPSIS

int \$FORMAT23
FileMem_oPrintf(*FileMem self, char *format, . . .*);

DESCRIPTION

Printf style function for FileMem objects. Returns the number of bytes printed.

NAME

Float_NewBlock — block allocator

SYNOPSIS

Float
Float_NewBlock(*Float self, int count, pointer address*);

DESCRIPTION

Returns a new Float arrayed object containing 'count' floats loaded from 'address'. Note: The proto-type (self) argument is not used and may be 0/NULL.

NAME

Float_Value — create a Float object

SYNOPSIS

Float
Float_Value(*float value*);

DESCRIPTION

Returns a new Float object with a single value.

NAME

HugeUID_Value — create a HugeUID object

SYNOPSIS

HugeUID
HugeUID_Value(*huge value*);

DESCRIPTION

Returns a new HugeUID object with a single value.

NAME

Huge_NewBlock — block allocator, ignores proto

SYNOPSIS

Huge

Huge_NewBlock(*Huge self, int count, pointer address*);

DESCRIPTION

Returns a new Huge arrayed object containing 'count' huge values loaded from 'address'.

NAME

Huge_Value — create a Huge object

SYNOPSIS

Huge

Huge_Value(*huge value*);

DESCRIPTION

Returns a new Huge object with a single value.

NAME

Int4_NewBlock — block allocator

SYNOPSIS

Int4

Int4_NewBlock(*Int4 self, int count, pointer address*);

DESCRIPTION

Returns a new Int4 arrayed object containing 'count' int4 values loaded from 'address'.

NAME

Int4_Value — create an Int4 object

SYNOPSIS

Int4

Int4_Value(*int4 value*);

DESCRIPTION

Returns a new Int4 object with a single value.

NAME

Int4_length — count of int4s before zero terminator. Note: Input NULL returns 0.

SYNOPSIS

```
int
Int4_length(Int4 self);
```

DESCRIPTION

Return the number of items before a null/zero int4 element. Useful for UTF-32 processing.

NAME

Json_FromBlock — create plist from data block

SYNOPSIS

```
Obj
Json_FromBlock(int count, pointer block, int flags);
```

DESCRIPTION

Returns a new plist from a block of data or NULL.

NAME

0Error — get last recorded error as Char or NULL

SYNOPSIS

```
Char
0Error(void);
```

DESCRIPTION

Returns the last error as a Char string which may be the 'description' key of a Dictionary.

NAME

0ErrorObject — get the last error object //

SYNOPSIS

```
Obj
0ErrorObject(void);
```

DESCRIPTION

Returns the last recorded error object or NULL. Note: Error objects are stored per thread.

NAME

0ErrorSet — takes ownership or nil

SYNOPSIS

```
Obj
0ErrorSet(Obj $CONSUMED obj);
```

DESCRIPTION

Set the current per thread error object, freeing the previous one, and always return 0/NULL. Note: The object should be a new error object. Error objects are either Char or Dictionary with a 'description' key.

NAME

PList_BinaryLoad — load an APPLE binary plist

SYNOPSIS

Obj
PList_BinaryLoad(*int count, pointer block*);

DESCRIPTION

Returns a PList object stored in an APPLE binary plist from in block of memory, or NULL. Note: Some binary elements are not supported such as SETS.

NAME

PList_FromBlock — create plist from data block

SYNOPSIS

Obj
PList_FromBlock(*int count, pointer block, int flags*);

DESCRIPTION

Returns a new plist from a block of data or NULL.

NAME

PList_Load — create plist from file.

SYNOPSIS

Obj
PList_Load(*char *file, int flags*);

DESCRIPTION

Returns a new PList object loaded from a file or NULL on error. See: OError(). Note: A PList object may be a Dictionary, Array, or any other Property List supported object. Formats: Apple property list, Coda-C property list, or binary PList with additional plugins.

NAME

PList_save — process to memory then save, safer.

SYNOPSIS

Obj
PList_save(*char *file, Obj container, int flags*);

DESCRIPTION

Save a property list to a file after the successful processing of the output data. See: PLIST_* flags for options. Returns NULL on error. See: OError(). Note: Currently does not support writing Apple binary plist.

EXAMPLES

```
if (!Plist_save(file,dict,PLIST_Apple)) { error processing... }
```

NAME

Real_NewBlock — block allocator, ignores prototype

SYNOPSIS

Real

Real_NewBlock(*Real self, int count, pointer address*);

DESCRIPTION

Returns a new Real arrayed object containing 'count' doubles loaded from 'address'.

NAME

Real_Value — create a Real object

SYNOPSIS

Real

Real_Value(*double value*);

DESCRIPTION

Returns a new Real object with a single value.

NAME

Short_NewBlock — block allocator

SYNOPSIS

Short

Short_NewBlock(*Short self, int count, pointer address*);

DESCRIPTION

Returns a new Short arrayed object containing 'count' shorts loaded from 'address'.

NAME

Short_Value — create a Short object

SYNOPSIS

Short

Short_Value(*short value*);

DESCRIPTION

Returns a new Short object with a single value.

NAME

Short_length — count of shorts before zero terminator. Note: Input NULL returns 0.

SYNOPSIS

```
int  
Short_length(Short self);
```

DESCRIPTION

Return the number of items before a null/zero short element. Useful for UTF-16 processing.

NAME

cc_count — Count # of c's in str[]

SYNOPSIS

```
int  
cc_count(char c, char *str);
```

DESCRIPTION

Return the number of occurrences of character in a string.

NAME

cc_is09 — is digit [0-9] \$S

SYNOPSIS

```
bool  
cc_is09(int c);
```

DESCRIPTION

Return true if character is a numerical digit.

NAME

cc_isAlpha — is letter [a-zA-Z] \$S

SYNOPSIS

```
bool  
cc_isAlpha(int c);
```

DESCRIPTION

Return true if character is an ASCII letter.

NAME

cc_isCap — is capital character [A-Z] \$S

SYNOPSIS

```
bool  
cc_isCap(int c);
```

DESCRIPTION

Return true if character is upper case ASCII letter.

NAME

cc_isLow — is lowercase character [a-z] \$S

SYNOPSIS

bool
cc_isLow(*int c*);

DESCRIPTION

Return true if character is lower case ASCII letter.

NAME

cc_isWhite — is white space character: sp,tab,nl,cr,ff \$S

SYNOPSIS

bool
cc_isWhite(*int c*);

DESCRIPTION

Return true if character is a white space character.

NAME

cc_pos — Position of c in str[] or -1 \$S

SYNOPSIS

int
cc_pos(*char c, const char *str*);

DESCRIPTION

Return the position of the first occurrence of a character in a string or EOF(-1).

NAME

cc_posCap — Position of c in str[], ignoring Case \$S

SYNOPSIS

int
cc_posCap(*char c, const char *str*);

DESCRIPTION

Return the position of the first occurrence of a character, ignoring case, in a string or EOF(-1).

NAME

cc_revPos — Position of c in str[], searching backwards (reverse) \$S

SYNOPSIS

int
cc_revPos(*char c, const char *str*);

DESCRIPTION

Return the position of the last occurrence of a character in a string or EOF(-1).

NAME

cc_toCap — Lowercase to uppercase \$S

SYNOPSIS

```
int  
cc_toCap(int c);
```

DESCRIPTION

Return input character in uppercase. Only [a-z] are changed.

NAME

cc_toLow — Uppercase to lowercase \$S

SYNOPSIS

```
int  
cc_toLow(int c);
```

DESCRIPTION

Return input character in lowercase. Only [A-Z] are changed.

NAME

cs_cap — capitalize a string, passthru

SYNOPSIS

```
char*  
cs_cap(char *str);
```

DESCRIPTION

Capitalize an ASCII string. Returns the original string.

NAME

cs_ccChange — change all f's to t's.

SYNOPSIS

```
void  
cs_ccChange(char *str, char f, char t);
```

DESCRIPTION

Change all 'from' characters to 'to' characters in a string.

NAME

cs_copy — Copy strings. Note: strings can overlap. \$S

SYNOPSIS

```
void  
cs_copy(char *dest, const char *sour);
```

DESCRIPTION

Copy source string to destination string. Note: The source and destination may overlap but if used incorrectly can cause a program crash.

NAME

cs_exactCap — exact compare strings, ignoring case \$S

SYNOPSIS

```
bool  
cs_exactCap(const char *str1, const char *str2);
```

DESCRIPTION

Returns true if two strings are the same, ignoring case.

NAME

cs_leftTrim — remove white from the start of string \$S

SYNOPSIS

```
void  
cs_leftTrim(char *str);
```

DESCRIPTION

Remove leading white characters from a string.

NAME

cs_low — lowercase a string, passthru

SYNOPSIS

```
char*  
cs_low(char *str);
```

DESCRIPTION

Lowercases an ASCII string. Returns the original string.

NAME

cs_pos — Position of key[] in str[] \$S

SYNOPSIS

```
int  
cs_pos(const char *key, const char *str);
```

DESCRIPTION

Returns the position of 'key' string in given string or EOF(-1).

NAME

cs_posCap — Position of key[] in str[], ignoring case \$S

SYNOPSIS

int
cs_posCap(*const char *key, const char *str*);

DESCRIPTION

Returns the position of 'key' string in given string, ignoring case, or EOF(-1).

NAME

cs_prefix — does str[] start with key[]? \$S

SYNOPSIS

bool
cs_prefix(*const char *key, const char *str*);

DESCRIPTION

Returns true if the 'key' string occurs at the start of the given string.

NAME

cs_prefixCap — does str[] start with key[] ignoring case? \$S

SYNOPSIS

bool
cs_prefixCap(*const char *key, const char *str*);

DESCRIPTION

Returns true if the 'key' string occurs at the start of the given string, ignoring case.

NAME

cs_reverse — reverse a string in place // zstrev()

SYNOPSIS

void
cs_reverse(*char *a*);

DESCRIPTION

Reverse a C string in place.

NAME

cs_rightTrim — remove white from the end of string \$S

SYNOPSIS

```
void  
cs_rightTrim(char *str);
```

DESCRIPTION

Remove trailing white characters from a string.

NAME

cs_rmc — remove a character from a string, return # removed.

SYNOPSIS

```
int  
cs_rmc(char *a, char c);
```

DESCRIPTION

Removes all occurrences of character from an string, returning the number removed.

NAME

cs_suffix — does str[] end with key[]? \$S

SYNOPSIS

```
bool  
cs_suffix(const char *key, const char *str);
```

DESCRIPTION

Returns true if the 'key' string occurs at the end of the given string.

NAME

cs_suffixCap — does str[] end with key[] ignoring case? \$S

SYNOPSIS

```
bool  
cs_suffixCap(const char *key, const char *str);
```

DESCRIPTION

Returns true if the 'key' string occurs at the end of the given string, ignoring case.

NAME

cs_toInt4 — convert string to four byte integer \$S

SYNOPSIS

```
int4  
cs_toInt4(const char *string);
```

DESCRIPTION

Convert string to four byte integer.

SEE ALSO

strtol(3)

NAME

cs_toReal — convert string to double \$S

SYNOPSIS

double

cs_toReal(*const char *string*);

DESCRIPTION

Returns a double value from a string using strtod().

SEE ALSO

strtod(3)

NAME

cs_trim — remove white from start and end of string \$S

SYNOPSIS

void

cs_trim(*char *str*);

DESCRIPTION

Remove leading and trailing white characters from a string.

NAME

file_Folder — directory entries as Named Array of <Char>. or nil

SYNOPSIS

Array

file_Folder(*char *path*);

DESCRIPTION

Returns an Array of Char strings for the entries in a directory. The Array's 'name' property is set to 'path'. Note: If the folder can't be opened, NULL is returned. Also the entries '.' and '..' are not included.

NAME

file_getCs — get string from FILE, max>1, max==sizeof(s) \$S

SYNOPSIS

int

file_getCs(*FILE *f, char *s, int mx*);

DESCRIPTION

Read a line from a file returning the number of characters loaded in the given buffer or EOF(-1). The maximum is normally the sizeof(buffer).

NAME

file_size — get file size, EOF on error

SYNOPSIS

huge
file_size(*char *path*);

DESCRIPTION

Returns the size of a file or EOF(-1) on error.

NAME

file_time — get file time (raw), EOF on error

SYNOPSIS

huge
file_time(*char *path*);

DESCRIPTION

Returns the modification time of a file as reported by 'stat()' or EOF(-1) on error.

NAME

file_typeCode — get the Coda-C path type code

SYNOPSIS

int
file_typeCode(*char *path, bool reportlinks*);

DESCRIPTION

Returns a printable character for the type of a path. Codes are like those reported by 'ls -l'. If 'reportlinks' is set then links are reported as 'l'. Errors are reported as '?' and normally indicates that the file is missing. Supports file_type() and file_IsType() macros.

SEE ALSO

file_type(3), file_IsType(3)

NAME

pointer_sort — function based sort

SYNOPSIS

void
pointer_sort(*pointer base, int nel, void *IfunVVC, void *context*);

DESCRIPTION

Sort an array of pointers with: `int (*compare)(Obj a,Obj b,context)`.

EXAMPLES

```
pointer_sort(vector,nel,strcmp,0); // to sort vector of strings.
```

NAME

u2_ToU4 — create UTF32 Int4 object from UTF-16.

SYNOPSIS

Int4

u2_ToU4(*pointer u2fstr, int countOrNeg1, int *faults*);

DESCRIPTION

unicode conversion function.

NAME

u2_ToUc — create UTF8 Char object from UTF-16.

SYNOPSIS

Char

u2_ToUc(*pointer u2str, int countOrNeg1, int *faults*);

DESCRIPTION

unicode conversion function.

NAME

u2_codes — # of unicodes (including faults, aka short*)

SYNOPSIS

int

u2_codes(*pointer u2str*);

DESCRIPTION

Returns the number of codepoints in a UTF-16 string.

NAME

u2_count — number of shorts for N unicodes. where $N \leq \text{u2_codes}(u2str)$;

SYNOPSIS

int

u2_count(*pointer u2str, int codes*);

DESCRIPTION

Returns the number of shorts in a UTF-16 string for the given number of codepoints.

NAME

u2_encode — encode UTF-16, ptr must be even address!

SYNOPSIS

pointer

u2_encode(*pointer ptr, int4 value, pointer context*);

DESCRIPTION

UTF-16 output encoder function used with uN_Covert() in machine endian format.

NAME

u2_scan — ret(#unicodes),count,faults

SYNOPSIS

int

u2_scan(*pointer u2str, int countOrNeg1, int codes, int *counter, int *faults*);

DESCRIPTION

UTF-16 string scanner. Returns number of codepoints, number of bytes in 'counter' and faults.

NAME

u4_ToU2 — create UTF16 Short object from UTF-32.

SYNOPSIS

Short

u4_ToU2(*pointer u4str, int countOrNeg1, int *faults*);

DESCRIPTION

unicode conversion function.

NAME

u4_ToUc — create UTF8 Char object from UTF-32.

SYNOPSIS

Char

u4_ToUc(*pointer u4str, int countOrNeg1, int *faults*);

DESCRIPTION

unicode conversion function.

NAME

u4_encode — encode UTF-32, (ptr aka Int4*)

SYNOPSIS

pointer

u4_encode(*pointer ptr, int4 value, pointer context*);

DESCRIPTION

UTF-32 output encoder function used with `uN_Convert()` in machine endian format.

NAME

`uN_Convert` — X

SYNOPSIS

Obj

```
uN_Convert(int isizeN, UcEncoder encoder, pointer context, pointer uNstr,
           int countOrNeg1, int *faults);
```

DESCRIPTION

General unicode conversion function between UTF-8, UTF-16 and UTF-32.

RETURN VALUES

Returns a new unicode object (Char, Short, or Int4)

NAME

`uc_ToU2` — create UTF16 Short object from UTF-8.

SYNOPSIS

Short

```
uc_ToU2(char *ustr, int countOrNeg1, int *faults);
```

DESCRIPTION

unicode conversion function.

NAME

`uc_ToU4` — create UTF32 Int4 object from UTF-8.

SYNOPSIS

Int4

```
uc_ToU4(char *ustr, int countOrNeg1, int *faults);
```

DESCRIPTION

unicode conversion function.

NAME

`uc_codes` — # of unicodes (including faults).

SYNOPSIS

int

```
uc_codes(char* ustr);
```

DESCRIPTION

Returns the number of codepoints in a UTF-8 string.

NAME

uc_count — # of bytes for N codes. where $N \leq \text{uc_codes}(\text{ustr})$;

SYNOPSIS

int
uc_count(*char *ustr, int codes*);

DESCRIPTION

Returns the number of bytes for a given number of codepoints in a UTF-8 string.

NAME

uc_encode — encode UTF-8

SYNOPSIS

pointer
uc_encode(*pointer ptr, int4 value, pointer context*);

DESCRIPTION

UTF-8 output encoder function used with `uN_Covert()`.

NAME

uc_faults — check string for UTF-8 faults

SYNOPSIS

int
uc_faults(*char *ustr*);

DESCRIPTION

Returns the number of faults in a null terminated UTF-8 string.

NAME

uc_scan — ret(#unicodes),count,faults

SYNOPSIS

int
uc_scan(*char *ustr, int lenOrNeg1, int codes, int *counter, int *faults*);

DESCRIPTION

UTF-8 string scanner. Returns number of codepoints, number of bytes in 'counter' and faults.